

for more details on BSP trees.

11.2.7.5 Choosing a Scene Graph

Clearly there are many different kinds of scene graphs. Which data structure to select for your game will depend upon the nature of the scenes you expect to be rendering. To make the choice wisely, you must have a clear understanding of what is required—and more importantly what is *not* required—when rendering scenes for your particular game.

For example, if you're implementing a fighting game, in which two characters battle it out in a ring surrounded by a mostly static environment, you may not need much of a scene graph at all. If your game takes place primarily in enclosed indoor environments, a BSP tree or portal system may serve you well. If the action takes place outdoors on relatively flat terrain, and the scene is viewed primarily from above (as might be the case in a strategy game or god game), a simple quadtree might be all that's required to achieve high rendering speeds. On the other hand, if an outdoor scene is viewed primarily from the point of view of someone on the ground, we may need additional culling mechanisms. Densely populated scenes can benefit from an occlusion volume (antiportal) system, because there will be plenty of occluders. On the other hand, if your outdoor scene is very sparse, adding an antiportal system probably won't pay dividends (and might even hurt your frame rate).

Ultimately, your choice of scene graph should be based on hard data obtained by actually measuring the performance of your rendering engine. You may be surprised to learn where all your cycles are actually going! But once you know, you can select scene graph data structures and/or other optimizations to target the specific problems at hand.

11.3 Advanced Lighting and Global Illumination

In order to render photorealistic scenes, we need physically accurate global illumination algorithms. A complete coverage of these techniques is beyond our scope. In the following sections, we will briefly outline the most prevalent techniques in use within the game industry today. Our goal here is to provide you with an awareness of these techniques and a jumping-off point for further investigation. For an excellent in-depth coverage of this topic, see [10].

11.3.1 Image-Based Lighting

A number of advanced lighting and shading techniques make heavy use of image data, usually in the form of two-dimensional texture maps. These are

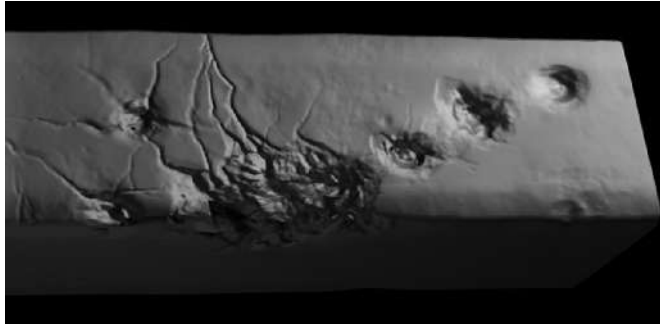


Figure II.52. An example of a normal-mapped surface.

called *image-based lighting* algorithms.

II.3.1.1 Normal Mapping

A *normal map* specifies a surface normal direction vector at each texel. This allows a 3D modeler to provide the rendering engine with a highly detailed description of a surface's shape, without having to tessellate the model to a high degree (as would be required if this same information were to be provided via vertex normals). Using a normal map, a single flat triangle can be made to look as though it were constructed from millions of tiny triangles. An example of normal mapping is shown in Figure 11.52.

The normal vectors are typically encoded in the RGB color channels of the texture, with a suitable bias to overcome the fact that RGB channels are strictly positive while normal vector components can be negative. Sometimes only two coordinates are stored in the texture; the third can be easily calculated at runtime, given the assumption that the surface normals are unit vectors.

II.3.1.2 Heightmaps: Bump, Parallax and Displacement Mapping

As its name implies, a *heightmap* encodes the height of the ideal surface above or below the surface of the triangle. Heightmaps are typically encoded as grayscale images, since we only need a single height value per texel. Heightmaps can be used for *bump mapping*, *parallax occlusion mapping* and *displacement mapping*—three techniques that can make a planar surface appear to have height variation.

In bump mapping, a heightmap is used as a cheap way to generate surface normals. This technique was primarily used in the early days of 3D graphics—nowadays, most game engines store surface normal information



Figure 11.53. Comparison of bump mapping (left), parallax occlusion mapping (center) and displacement mapping (right).

explicitly in a *normal map*, rather than calculating the normals from a heightmap.

Parallax occlusion mapping uses the information in a heightmap to artificially adjust the texture coordinates used when rendering a flat surface, in such a way as to make the surface appear to contain surface details that move semi-correctly as the camera moves. (This technique was used to produce the bullet impact decals in the *Uncharted* series of games by Naughty Dog.)

Displacement mapping (also known as relief mapping) produces real surface details by actually tessellating and then extruding surface polygons, again using a heightmap to determine how much to displace each vertex. This produces the most convincing effect—one that properly self-occludes and self-shadows—because real geometry is being generated. Figure 11.53 compares bump mapping, parallax mapping and displacement mapping. Figure 11.54 shows an example of displacement mapping implemented in DirectX 9.

11.3.1.3 Specular/Gloss Maps

When light reflects directly off a shiny surface, we call this *specular* reflection. The intensity of a specular reflection depends on the relative angles of the viewer, the light source and the surface normal. As we saw in Section 11.1.3.2, the specular intensity takes the form $k_S(\mathbf{R} \cdot \mathbf{V})^\alpha$, where $\mathbf{R}$ is the reflection of the light's direction vector about the surface normal, $\mathbf{V}$ is the direction to the viewer, k_S is the overall specular reflectivity of the surface and α is called the specular power.

Many surfaces aren't uniformly glossy. For example, when a person's face is sweaty and dirty, wet regions appear shiny, while dry or dirty areas appear dull. We can encode high-detail specular information in a special texture map known as a *specular map*.

If we store the value of k_S in the texels of a specular map, we can control how much specular reflection should be applied at each texel. This kind of specular map is sometimes called a *gloss map*. It is also called a *specular mask*, because zero-valued texels can be used to “mask off” regions of the surface where we do not want specular reflection applied. If we store the value of



Figure II.54. DirectX 9 displacement mapping. Simple source geometry is tessellated at runtime to produce the surface details.

α in our specular map, we can control the amount of “focus” our specular highlights will have at each texel. This kind of texture is called a *specular power map*. An example of a gloss map is shown in Figure 11.55.

II.3.1.4 Environment Mapping

An environment map looks like a panoramic photograph of the environment taken from the point of view of an object in the scene, covering a full 360 degrees horizontally and either 180 degrees or 360 degrees vertically. An environment map acts like a description of the general lighting environment surrounding an object. It is generally used to inexpensively render reflections.

The two most common formats are *spherical environment maps* and *cubic environment maps*. A spherical map looks like a photograph taken through a fisheye lens, and it is treated as though it were mapped onto the inside of a sphere whose radius is infinite, centered about the object being rendered. The problem with sphere maps is that they are addressed using spherical coordinates. Around the equator, there is plenty of resolution both horizontally and vertically. However, as the vertical (azimuthal) angle approaches vertical, the resolution of the texture along the horizontal (zenith) axis decreases to a single



Figure 11.55. This screenshot from EA's *Fight Night Round 3* shows how a gloss map can be used to control the degree of specular reflection that should be applied to each texel of a surface. (See Color Plate XX.)

texel. Cube maps were devised to avoid this problem.

A cube map looks like a composite photograph pieced together from photos taken in the six primary directions (up, down, left, right, front and back). During rendering, a cube map is treated as though it were mapped onto the six inner surfaces of a box at infinity, centered on the object being rendered.

To read the environment map texel corresponding to a point P on the surface of an object, we take the ray from the camera to the point P and reflect it about the surface normal at P . The reflected ray is followed until it intersects the sphere or cube of the environment map. The value of the texel at this intersection point is used when shading the point P .

11.3.1.5 Three-Dimensional Textures

Modern graphics hardware also includes support for three-dimensional textures. A 3D texture can be thought of as a stack of 2D textures. The GPU knows how to address and filter a 3D texture, given a three-dimensional texture coordinate (u, v, w) .

Three-dimensional textures can be useful for describing the appearance or volumetric properties of an object. For example, we could render a marble sphere and allow it to be cut by an arbitrary plane. The texture would look continuous and correct across the cut no matter where it was made, because the texture is well-defined and continuous throughout the entire volume of the sphere.

11.3.2 High Dynamic Range Lighting

A display device like a television set or CRT monitor can only produce a limited range of intensities. This is why the color channels in the frame buffer are limited to a zero to one range. But in the real world, light intensities can grow arbitrarily large. *High dynamic range* (HDR) lighting attempts to capture this wide range of light intensities.

HDR lighting performs lighting calculations without clamping the resulting intensities arbitrarily. The resulting image is stored in a format that permits intensities to grow beyond one. The net effect is an image in which extreme dark and light regions can be represented without loss of detail within either type of region.

Prior to display on-screen, a process called *tone mapping* is used to shift and scale the image's intensity range into the range supported by the display device. Doing this permits the rendering engine to reproduce many real-world visual effects, like the temporary blindness that occurs when you walk from a dark room into a brightly lit area, or the way light seems to bleed out from behind a brightly back-lit object (an effect known as *bloom*).

One way to represent an HDR image is to store the R, G and B channels using 32-bit floating-point numbers, instead of 8-bit integers. Another alternative is to employ an entirely different color model altogether. The log-LUV color model is a popular choice for HDR lighting. In this model, color is represented as an intensity channel (L) and two chromaticity channels (U and V). Because the human eye is more sensitive to changes in intensity than it is to changes in chromaticity, the L channel is stored in 16 bits while U and V are given only eight bits each. In addition, L is represented using a logarithmic scale (base two) in order to capture a very wide range of light intensities.

11.3.3 Global Illumination

As we noted in Section 11.1.3.1, global illumination refers to a class of lighting algorithms that account for light's interactions with multiple objects in the scene, on its way from the light source to the virtual camera. Global illumination accounts for effects like the shadows that arise when one surface occludes another, reflections, caustics and the way the color of one object can "bleed" onto the objects around it. In the following sections, we'll take a brief look at some of the most common global illumination techniques. Some of these methods aim to reproduce a single isolated effect, like shadows or reflections. Others like radiosity and ray tracing methods aim to provide a holistic model of global light transport.

11.3.3.1 Shadow Rendering

Shadows are created when a surface blocks light's path. The shadows caused by an ideal point light source would be sharp, but in the real world shadows have blurry edges; this is called the penumbra. A penumbra arises because real-world light sources cover some area and so produce light rays that graze the edges of an object at different angles.

The two most prevalent shadow rendering techniques are *shadow volumes* and *shadow maps*. We'll briefly describe each in the sections below. In both techniques, objects in the scene are generally divided into three categories: objects that cast shadows, objects that are to receive shadows and objects that are entirely excluded from consideration when rendering shadows. Likewise, the lights are tagged to indicate whether or not they should generate shadows. This important optimization limits the number of light-object combinations that need to be processed in order to produce the shadows in a scene.

Shadow Volumes

In the shadow volume technique, each shadow caster is viewed from the vantage point of a shadow-generating light source, and the shadow caster's silhouette edges are identified. These edges are extruded in the direction of the light rays emanating from the light source. The result is a new piece of geometry that describes the volume of space in which the light is occluded by the shadow caster in question. This is shown in Figure 11.56.



Figure 11.56. A shadow volume generated by extruding the silhouette edges of a shadow casting object as seen from the point of view of the light source.

A shadow volume is used to generate a shadow by making use of a special full-screen buffer known as the stencil buffer. This buffer stores a single integer value corresponding to each pixel of the screen. Rendering can be masked by the values in the stencil buffer—for example, we could configure the GPU to only render fragments whose corresponding stencil values are nonzero. In addition, the GPU can be configured so that rendered geometry updates the values in the stencil buffer in various useful ways.

To render shadows, the scene is first drawn to generate an unshadowed image in the frame buffer, along with an accurate z-buffer. The stencil buffer is cleared so that it contains zeros at every pixel. Each shadow volume is then rendered from the point of view of the camera in such a way that front-facing triangles increase the values in the stencil buffer by one, while back-facing triangles decrease them by one. In areas of the screen where the shadow volume does not appear at all, of course the stencil buffer's pixels will be left containing zero. The stencil buffer will also contain zeros where both the front and back faces of the shadow volume are visible, because the front face will increase the stencil value but the back face will decrease it again. In areas where the back face of the shadow volume has been occluded by “real” scene geometry, the stencil value will be one. This tells us which pixels of the screen are in shadow. So we can render shadows in a third pass, by simply darkening those regions of the screen that contain a nonzero stencil buffer value.

Shadow Maps

The shadow mapping technique is effectively a per-fragment depth test performed from the point of view of the light instead of from the point of view of the camera. The scene is rendered in two steps: First, a *shadow map* texture is generated by rendering the scene from the point of view of the light source and saving off the contents of the depth buffer. Second, the scene is rendered as usual, and the shadow map is used to determine whether or not each fragment is in shadow. At each fragment in the scene, the shadow map tells us whether or not the light is being occluded by some geometry that is closer to the light source, in just the same way that the z-buffer tells us whether a fragment is being occluded by a triangle that is closer to the camera.

A shadow map contains only depth information—each texel records how far away it is from the light source. Shadow maps are therefore typically rendered using the hardware's double-speed z-only mode (since all we care about is the depth information). For a point light source, a perspective projection is used when rendering the shadow map; for a directional light source, an orthographic projection is used instead.

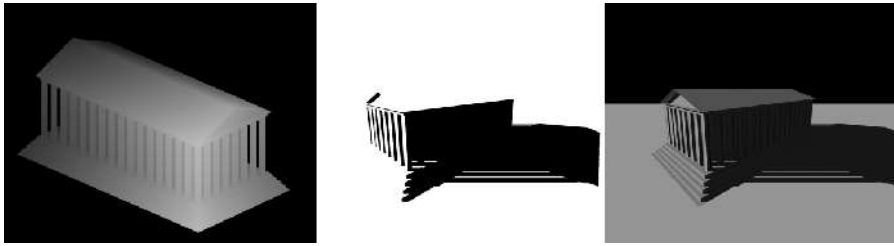


Figure 11.57. The far left image is a shadow map—the contents of the z-buffer as rendered from the point of view of a particular light source. The pixels of the center image are black where the light-space depth test failed (fragment in shadow) and white where it succeeded (fragment not in shadow). The far right image shows the final scene rendered with shadows.

To render a scene using a shadow map, we draw the scene as usual from the point of view of the camera. For each vertex of every triangle, we calculate its position in *light space*—i.e., in the same “view space” that was used when generating the shadow map in the first place. These light-space coordinates can be interpolated across the triangle, just like any other vertex attribute. This gives us the position of each fragment *in light space*. To determine whether a given fragment is in shadow or not, we convert the fragment’s light-space (x, y) -coordinates into texture coordinates (u, v) within the shadow map. We then compare the fragment’s light-space z -coordinate with the depth stored at the corresponding texel in the shadow depth map. If the fragment’s light-space z is farther away from the light than the texel in the shadow map, then it must be occluded by some other piece of geometry that is closer to the light source—hence it is in shadow. Likewise, if the fragment’s light-space z is closer to the light source than the texel in the shadow map, then it is not occluded and is not in shadow. Based on this information, the fragment’s color can be adjusted accordingly. The shadow mapping process is illustrated in Figure 11.57.

11.3.3.2 Ambient Occlusion

Ambient occlusion is a technique for modeling *contact shadows*—the soft shadows that arise when a scene is illuminated by only ambient light. In effect, ambient occlusion describes how “accessible” each point on a surface is to light in general. For example, the interior of a section of pipe is less accessible to ambient light than its exterior. If the pipe were placed outside on an overcast day, its interior would generally appear darker than its exterior.

Figure 11.58 shows how ambient occlusion produces shadows underneath a car and in its wheel wells, as well as within the seams between body panels. Ambient occlusion is measured at a point on a surface by constructing



Figure 11.58. A car rendered with ambient occlusion. Notice the darkened areas underneath the vehicle and in the wheel wells.

a hemisphere with a very large radius centered on that point and determining what percentage of that hemisphere's area is visible from the point in question. It can be precomputed offline for static objects, because ambient occlusion is independent of view direction and the direction of incident light. It is typically stored in a texture map that records the level of ambient occlusion at each texel across the surface.

11.3.3.3 Reflections

Reflections occur when light bounces off a highly specular (shiny) surface producing an image of another portion of the scene in the surface. Reflections can be implemented in a number of ways. Environment maps are used to produce general reflections of the surrounding environment on the surfaces of shiny objects. Direct reflections in flat surfaces like mirrors can be produced by reflecting the camera's position about the plane of the reflective surface and then rendering the scene from that reflected point of view into a texture. The texture is then applied to the reflective surface in a second pass (see Figure 11.59).

11.3.3.4 Caustics

Caustics are the bright specular highlights arising from intense reflections or refractions from very shiny surfaces like water or polished metal. When the reflective surface moves, as is the case for water, the caustic effects glimmer and "swim" across the surfaces on which they fall. Caustic effects can be produced by projecting a (possibly animated) texture containing semi-random bright highlights onto the affected surfaces. An example of this technique is shown in Figure 11.60.



Figure 11.59. Mirror reflections in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog. PlayStation 4) implemented by rendering the scene to a texture that is subsequently applied to the mirror's surface. (See Color Plate XXI.)

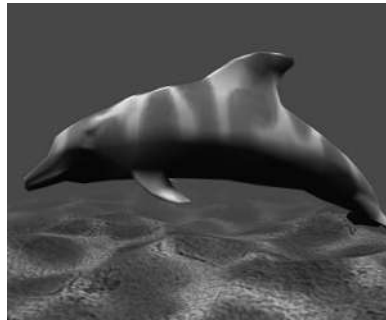


Figure 11.60. Water caustics produced by projecting an animated texture onto the affected surfaces.

11.3.3.5 Subsurface Scattering

When light enters a surface at one point, is scattered beneath the surface, and then reemerges at a different point on the surface, we call this *subsurface scattering*. This phenomenon is responsible for the “warm glow” of human skin, wax and marble statues (e.g., Figure 11.61). Subsurface scattering is described by a more-advanced variant of the BRDF (see Section 11.1.3.2) known as the BSSRDF (*bidirectional surface scattering reflectance distribution function*).

Subsurface scattering can be simulated in a number of ways. Depth-map-based subsurface scattering renders a shadow map (see Section 11.3.3.1), but instead of using it to determine which pixels are in shadow, it is used to measure how far a beam of light would have to travel in order to pass all the



Figure 11.61. On the left, a dragon rendered without subsurface scattering (i.e., using a BRDF lighting model). On the right, the same dragon rendered with subsurface scattering (i.e., using a BSSRDF model). Images rendered by Rui Wang at the University of Virginia.

way through the occluding object. The shadowed side of the object is then given an artificial diffuse lighting term whose intensity is inversely proportional to the distance the light had to travel in order to emerge on the opposite side of the object. This causes objects to appear to be glowing slightly on the side opposite to the light source but only where the object is relatively thin. For more information on subsurface scattering techniques, see http://http.developer.nvidia.com/GPUGems/gpugems_ch16.html.

11.3.3.6 Precomputed Radiance Transfer (PRT)

Precomputed radiance transfer (PRT) is a popular technique that attempts to simulate the effects of radiosity-based rendering methods in real time. It does so by precomputing and storing a complete description of how an incident light ray would interact with a surface (reflect, refract, scatter, etc.) when approaching from every possible direction. At runtime, the response to a particular incident light ray can be looked up and quickly converted into very accurate lighting results.

In general the light's response at a point on the surface is a complex function defined on a hemisphere centered about the point. A compact representation of this function is required to make the PRT technique practical. A common approach is to approximate the function as a linear combination of spherical harmonic basis functions. This is essentially the three-dimensional equivalent of encoding a simple scalar function $f(x)$ as a linear combination of shifted and scaled sine waves.

The details of PRT are far beyond our scope. For more information, see <http://web4.cs.ucl.ac.uk/staff/j.kautz/publications/prtSIG02.pdf>. PRT lighting techniques are demonstrated in a DirectX sample program available in the DirectX SDK—see <http://msdn.microsoft.com/en-us/library/bb147287.aspx> for more details.

11.3.4 Deferred Rendering

In traditional triangle-rasterization-based rendering, all lighting and shading calculations are performed on the triangle fragments in world space, view space or tangent space. The problem with this technique is that it is inherently inefficient. For one thing, we potentially do work that we don't need to do. We shade the vertices of triangles, only to discover during the rasterization stage that the entire triangle is being depth-culled by the z-test. Early z-tests help eliminate unnecessary pixel shader evaluations, but even this isn't perfect. What's more, in order to handle a complex scene with lots of lights, we end up with a proliferation of different versions of our vertex and pixel shaders—versions that handle different numbers of lights, different types of lights, different numbers of skinning weights, etc.

Deferred rendering is an alternative way to shade a scene that addresses many of these problems. In deferred rendering, the majority of the lighting calculations are done in screen space, not view space. We efficiently render the scene without worrying about lighting. During this phase, we store all the information we're going to need to light the pixels in a “deep” frame buffer known as the *G-buffer*. Once the scene has been fully rendered, we use the information in the G-buffer to perform our lighting and shading calculations. This is usually much more efficient than view-space lighting, avoids the proliferation of shader variants and permits some very pleasing effects to be rendered relatively easily.

The G-buffer may be physically implemented as a collection of buffers, but conceptually it is a single frame buffer containing a rich set of information about the lighting and surface properties of the objects in the scene at every pixel on the screen. A typical G-buffer might contain the following per-pixel attributes: depth, surface normal in view space or world space, diffuse color, specular power, even precomputed radiance transfer (PRT) coefficients. The following sequence of screenshots from Guerrilla Games' *Killzone 2* (Figure 11.62) shows some of the typical components of the G-buffer.

An in-depth discussion of deferred rendering is beyond our scope, but the folks at Guerrilla Games have prepared an excellent presentation on the topic, which is available at <http://www.slideshare.net/guerrillagames/deferred-rendering-in-killzone-2-9691589>.

11.3.5 Physically Based Shading

Traditional game lighting engines have required artists and lighters to tweak a wide variety of sometimes non-intuitive parameters, across numerous disparate rendering engine systems, in order to achieve a desired “look” in-game.

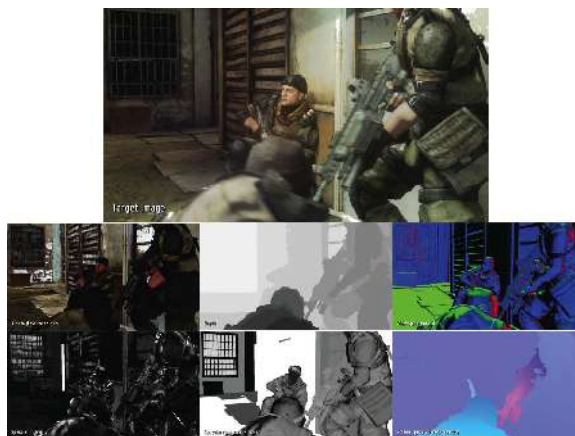


Figure 11.62. Screenshots from *Killzone 2* by Guerrilla Games, showing some of the typical components of the G-buffer used in deferred rendering. The upper image shows the final rendered image. Below it, clockwise from the upper left, are the albedo (diffuse) color, depth, view-space normal, screen-space 2D motion vector (for motion blurring), specular power and specular intensity. (See Color Plate XXII.)

This can be an arduous and time-consuming process. What's worse, parameter settings that work well under one set of lighting conditions might not work well under other lighting scenarios. To address these problems, rendering programmers are turning toward *physically based shading* models.

A physically based shading model attempts to approximate the ways in which light travels and interacts with materials in the real world, allowing artists and lighters to tweak shader parameters using intuitive, real-world quantities measured in real-world units. A complete discussion of physically based shading is beyond the scope of this book, but you can start to learn more about it here: <https://www.marmoset.co/toolbag/learn/pbr-theory>.

11.4 Visual Effects and Overlays

The rendering pipeline we've discussed to this point is responsible primarily for rendering three-dimensional solid objects. A number of specialized rendering systems are typically layered on top of this pipeline, responsible for rendering visual elements like particle effects, decals (small geometry overlays that represent bullet holes, cracks, scratches and other surface details), hair and fur, rain or falling snow, water and other specialized visual effects. Full-screen post effects may be applied, including vignette (a reduction of brightness and saturation around the edges of the screen), motion blur, depth

of field blurring, artificial/enhanced colorization, and the list goes on. Finally, the game's menu system and heads-up display (HUD) are typically realized by rendering text and other two- or three-dimensional graphics in screen space overlaid on top of the three-dimensional scene.

An in-depth coverage of these engine systems is beyond our scope. In the following sections, we'll provide a brief overview of these rendering systems, and point you in the direction of additional information.

11.4.1 Particle Effects

A particle rendering system is concerned with rendering amorphous objects like clouds of smoke, sparks, flame and so on. These are called *particle effects*. The key features that differentiate a particle effect from other kinds of renderable geometry are as follows:

- It is composed of a very *large number of relatively simple* pieces of geometry—most often simple cards called *quads*, composed of two triangles each.
- The geometry is often *camera-facing* (i.e., billboarded), meaning that the engine must take steps to ensure that the face normals of each quad always point directly at the camera's focal point.
- Its materials are almost always *semitransparent* or *translucent*. As such, particle effects have some stringent *rendering order* constraints that do not apply to the majority of opaque objects in a scene.
- Particles *animate* in a rich variety of ways. Their positions, orientations, sizes (scales), texture coordinates and many of their shader parameters vary from frame to frame. These changes are defined either by hand-authored animation curves or via procedural methods.
- Particles are typically *spawned and killed* continually. A particle emitter is a logical entity in the world that creates particles at some user-specified rate; particles are killed when they hit a predefined death plane, or when they have lived for a user-defined length of time, or as decided by some other user-specified criteria.

Particle effects could be rendered using regular triangle mesh geometry with appropriate shaders. However, because of the unique characteristics listed above, a specialized particle effect animation and rendering system is always used to implement them in a real production game engine. A few example particle effects are shown in Figure 11.63.



Figure 11.63. Flame, smoke and bullet tracer particle effects in *Uncharted 3: Drake's Deception* (© 2011/™ SIE. Created and developed by Naughty Dog, PlayStation 3). (See Color Plate XXIII.)

Particle system design and implementation is a rich topic that could occupy many chapters all on its own. For more information on particle systems, see [2, Section 10.7], [16, Section 20.5], [11, Section 13.7] and [12, Section 4.1.2].

11.4.2 Decals

A *decal* is a relatively small piece of geometry that is overlaid on top of the regular geometry in the scene, allowing the visual appearance of the surface to be modified dynamically. Examples include bullet holes, foot prints, scratches, cracks, etc.

The approach most often used by modern engines is to model a decal as a rectangular area that is to be projected along a ray into the scene. This gives rise to a rectangular prism in 3D space. Whatever surface the prism intersects first becomes the surface of the decal. The triangles of the intersected geometry are extracted and clipped against the four bounding planes of the decal's projected prism. The resulting triangles are texture-mapped with a desired decal texture by generating appropriate texture coordinates for each vertex. These texture-mapped triangles are then rendered over the top of the regular scene, often using parallax mapping to give them the illusion of depth and with a slight z-bias (usually implemented by shifting the near plane slightly) so they don't experience z-fighting with the geometry on which they are overlaid. The result is the appearance of a bullet hole, scratch or other kind of surface modification. Some bullet-hole decals are depicted in Figure 11.64.

For more information on creating and rendering decals, see [9, Section 4.8] and [32, Section 9.2].

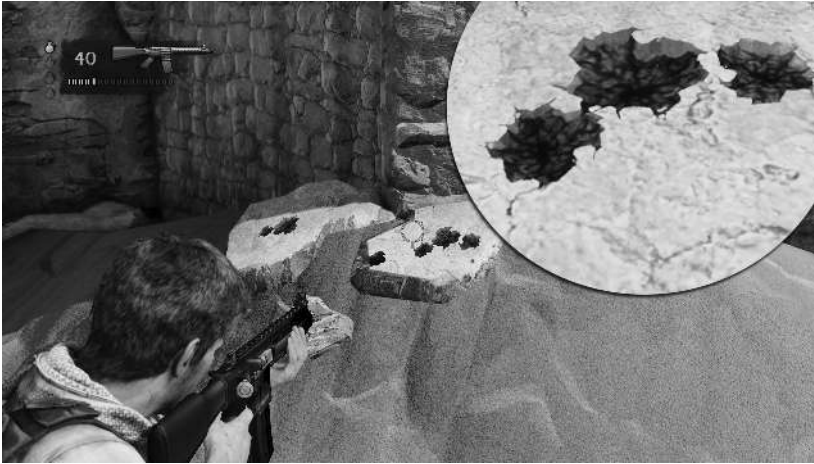


Figure 11.64. Parallax-mapped decals from *Uncharted 3: Drake's Deception* (© 2011/™ SIE. Created and developed by Naughty Dog, PlayStation 3). (See Color Plate XXIV.)

11.4.3 Environmental Effects

Any game that takes place in a somewhat natural or realistic environment requires some kind of environmental rendering effects. These effects are usually implemented via specialized rendering systems. We'll take a brief look at a few of the more common of these systems in the following sections.

11.4.3.1 Skies

The sky in a game world needs to contain vivid detail, yet technically speaking it lies an extremely long distance away from the camera. Therefore, we cannot model it as it really is and must turn instead to various specialized rendering techniques.

One simple approach is to fill the frame buffer with the sky texture prior to rendering any 3D geometry. The sky texture should be rendered at an approximate 1:1 texel-to-pixel ratio, so that the texture is roughly or exactly the resolution of the screen. The sky texture can be rotated and scrolled to correspond to the motions of the camera in-game. During rendering of the sky, we make sure to set the depth of all pixels in the frame buffer to the maximum possible depth value. This ensures that the 3D scene elements will always sort on top of the sky. The arcade hit *Hydro Thunder* rendered its skies in exactly this manner.

On modern game platforms, where pixel shading costs can be high, sky rendering is often done *after* the rest of the scene has been rendered. First the z-buffer is cleared to the maximum z-value. Then the scene is rendered.

Finally the sky is rendered, with z-testing enabled, z writing turned off, and using a z-test value that is one less than the maximum. This causes the sky to be drawn only where it is not occluded by closer objects like terrain, buildings and trees. Drawing the sky last ensures that its pixel shader is run for the minimum possible number of screen pixels.

For games in which the player can look in any direction, we can use a *sky dome* or *sky box*. The dome or box is rendered with its center always at the camera's current location, so that it appears to lie at infinity, no matter where the camera moves in the game world. As with the sky texture approach, the sky box or dome is rendered before any other 3D geometry, and all of the pixels in the frame buffer are set to the maximum z-value when the sky is rendered. This means that the dome or box can actually be tiny, relative to other objects in the scene. Its size is irrelevant, as long as it fills the entire frame buffer when it is drawn. For more information on sky rendering, see [2, Section 10.3] and [44, page 253].

Clouds are often implemented with a specialized rendering and animation system as well. In early games like *Doom* and *Quake*, the clouds were just planes with scrolling semitransparent cloud textures on them. More-recent cloud techniques include camera-facing cards (billboards), particle-effect based clouds and volumetric cloud effects.

II.4.3.2 Terrain

The goal of a terrain system is to model the surface of the earth and provide a canvas of sorts upon which other static and dynamic elements can be laid out. Terrain is sometimes modeled explicitly in a package like Maya. But if the player can see far into the distance, we usually want some kind of dynamic tessellation or other level of detail (LOD) system. We may also need to limit the amount of data required to represent very large outdoor areas.

Height field terrain is one popular choice for modeling large terrain areas. The data size can be kept relatively small because a height field is typically stored in a grayscale texture map. In most height-field-based terrain systems, the horizontal ($y = 0$) plane is tessellated in a regular grid pattern, and the heights of the terrain vertices are determined by sampling the height field texture. The number of triangles per unit area can be varied based on distance from the camera, thereby allowing large-scale features to be seen in the distance, while still permitting a good deal of detail to be represented for nearby terrain. An example of a terrain defined via a height field bitmap is shown in Figure 11.65.

Terrain systems usually provide specialized tools for “painting” the height field itself, carving out terrain features like roads, rivers and so on. Texture

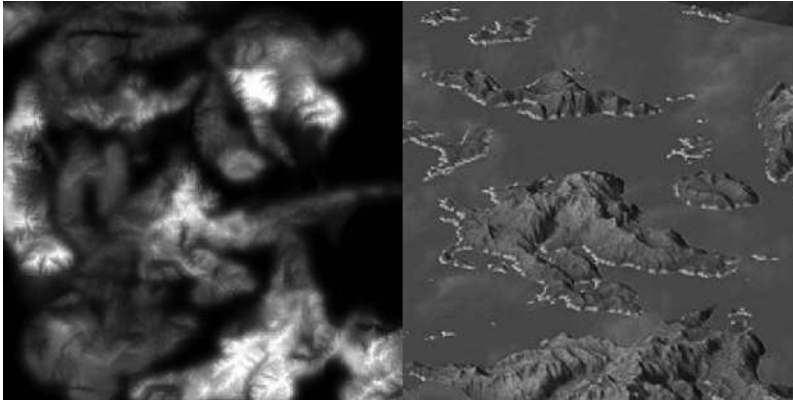


Figure 11.65. A grayscale height field bitmap (left) can be used to control the vertical positions of the vertices in a terrain grid mesh (right). In this example, a water plane intersects the terrain mesh to create islands.

mapping in a terrain system is often a blend between four or more textures. This allows artists to “paint” in grass, dirt, gravel and other terrain features by simply exposing one of the texture layers. The layers can be cross-blended from one to another to provide smooth textural transitions. Some terrain tools also permit sections of the terrain to be cut out to permit buildings, trenches and other specialized terrain features to be inserted in the form of regular mesh geometry. Terrain authoring tools are sometimes integrated directly into the game world editor, while in other engines they may be stand-alone tools.

Of course, height field terrain is just one of many options for modeling the surface of the Earth in a game. For more information on terrain rendering, see [8, Sections 4.16 through 4.19] and [9, Section 4.2].

11.4.3.3 Water

Water renderers are commonplace in games nowadays. There are lots of different kinds of water, including oceans, pools, rivers, waterfalls, fountains, jets, puddles and damp solid surfaces. Each type of water generally requires some specialized rendering technology. Some also require dynamic motion simulations. Large bodies of water may require dynamic tessellation or other LOD methodologies similar to those employed in a terrain system.

Water systems sometimes interact with a game’s rigid body dynamics system (flotation, force from water jets, etc.) and with gameplay (slippery surfaces, swimming mechanics, diving mechanics, riding vertical jets of water and so on). Water effects are often created by combining disparate rendering technologies and subsystems. For example, a waterfall might make use

of specialized water shaders, scrolling textures, particle effects for mist at the base, a decal-like overlay for foam, and the list goes on. Today's games offer some pretty amazing water effects, and active research into technologies like real-time fluid dynamics promises to make water simulations even richer and more realistic in the years ahead. For more information on water rendering and simulation techniques, see [2, Sections 9.3, 9.5 and 9.6], [15] and [8, Sections 2.6 and 5.11].

II.4.4 Overlays

Most games have heads-up displays, in-game graphical user interfaces and menu systems. These *overlays* are typically comprised of two- and three-dimensional graphics rendered directly in view space or screen space.

Overlays are generally rendered after the primary scene, with z-testing disabled to ensure that they appear on top of the three-dimensional scene. Two-dimensional overlays are typically implemented by rendering quads (triangle pairs) in screen space using an orthographic projection. Three-dimensional overlays may be rendered using an orthographic projection or via the regular perspective projection with the geometry positioned in view space so that it follows the camera around.

II.4.4.1 Text and Fonts

A game engine's text/font system is typically implemented as a special kind of two-dimensional (or sometimes three-dimensional) overlay. At its core, a text rendering system needs to be capable of displaying a sequence of character glyphs corresponding to a text string, arranged in various orientations on the screen.

A font is often implemented via a texture map known as a *glyph atlas*, which contains the various required glyphs. This texture typically consists of a single alpha channel—the value at each pixel representing the percentage of that pixel that is covered by the interior of a glyph. A font description file provides information such as the bounding boxes of each glyph within the texture, and font layout information such as kerning, baseline offsets and so on. A glyph is rendered by drawing a quad whose (u, v) coordinates correspond to the bounding box of the desired glyph within the atlas texture map. The texture map provides the alpha value, while the color is specified separately, allowing glyphs of any color to be rendered from the same atlas.

Another option for font rendering is to make use of a font library like FreeType (<https://www.freetype.org/>). The FreeType library enables a game or other application to read fonts in a wide variety of formats, including

TrueType (TTF) and OpenType (OTF), and to render glyphs into in-memory pixmaps at any desired point size. FreeType renders each glyph using its Bezier curve outlines, so it produces very accurate results.

Typically a real-time application like a game will use FreeType to prerender the necessary glyphs into an atlas, which is in turn used as a texture map to render glyphs as simple quads every frame. However, by embedding FreeType or a similar library in your engine, it's possible to render some glyphs into the atlas on the fly, on an as-needed basis. This can be useful when rendering text in a language with a very large number of possible glyphs, like Chinese or Korean.

Yet another way to render high-quality character glyphs is to use *signed distance fields* to describe the glyphs. In this approach, glyphs are rendered to pixmaps (as they would be with a library like FreeType), but the value at each pixel is no longer an alpha "coverage" value. Instead, each pixel contains a signed distance from that pixel center to the nearest edge of the glyph. Inside the glyph, the distances are negative; outside the glyph's outlines, they are positive. When rendering a glyph from a signed distance field texture atlas, the distances are used in the pixel shader to calculate highly accurate alpha values. The net result is text that looks smooth at any distance or viewing angle. You can read more about signed distance field text rendering by searching online for Konstantin Käfer's article entitled "Drawing Text with Signed Distance Fields in Mapbox GL," or the article written by Chris Green of Valve entitled "Improved Alpha-Tested Magnification for Vector Textures and Special Effects."

Glyphs can also be rendered directly from the Bézier curve outlines that define them. The Slug font rendering library by Terathon Software LLC performs its outline-based glyph rendering on the GPU, thereby making this technique practical for use in a real-time game application.

A good text/font system must account for the differences in character sets and reading directions inherent in various languages. Laying out the characters in a text string is a process known as *shaping* the string. The characters are laid out from left to right or right to left, depending on the language, with each character aligned to a common *baseline*. The spacing between characters is determined in part by metrics provided by the creator of the font (and stored in the font file), and partly by *Kerning* rules that dictate contextual intercharacter spacing adjustments.

Some text systems also provide various fun features like the ability to animate characters across the screen in various ways, the ability to animate individual characters and so on. However, it's important to remember when implementing a game font system that only those features that are *actually re-*

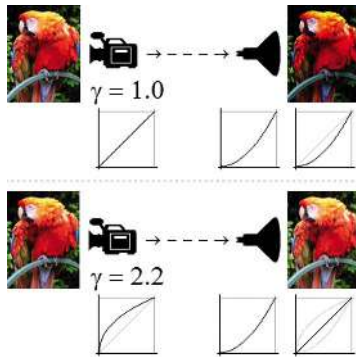


Figure 11.66. The effect of a CRT's gamma response on image quality and how the effect can be corrected for. Image courtesy of www.wikipedia.org.

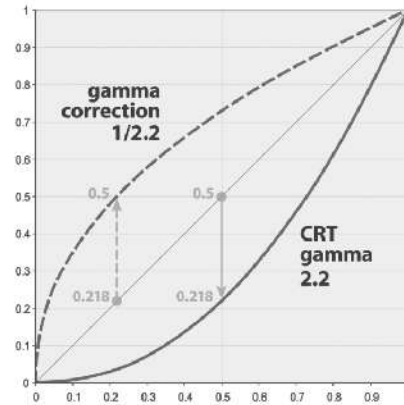


Figure 11.67. Gamma encoding and decoding curves. Image courtesy of www.wikipedia.org.

quired by the game should be implemented. There's no point in furnishing your engine with an advanced text animation if your game never needs to display animated text, for example.

11.4.5 Gamma Correction

CRT monitors tend to have a nonlinear response to luminance values. That is, if a linearly increasing ramp of R, G or B values were to be sent to a CRT, the image that would result on-screen would be perceptually nonlinear to the human eye. Visually, the dark regions of the image would look darker than they should. This is illustrated in Figure 11.66.

The gamma response curve of a typical CRT display can be modeled quite simply by the formula

$$V_{\text{out}} = V_{\text{in}}^{\gamma}$$

where $\gamma_{\text{CRT}} > 1$. To correct for this effect, the colors sent to the CRT display are usually passed through an inverse transformation (i.e., using a gamma value $\gamma_{\text{corr}} < 1$). The value of γ_{CRT} for a typical CRT monitor is 2.2, so the correction value is usually $\gamma_{\text{corr}} \approx 1/2.2 = 0.455$. These gamma encoding and decoding curves are shown in Figure 11.67.

Gamma encoding can be performed by the 3D rendering engine to ensure that the values in the final image are properly gamma-corrected. One problem that is encountered, however, is that the bitmap images used to represent texture maps are often gamma-corrected themselves. A high-quality rendering engine takes this fact into account, by gamma-decoding the textures prior to

rendering and then re-encoding the gamma of the final rendered scene so that its colors can be reproduced properly on-screen.

11.4.6 Full-Screen Post Effects

Full-screen *post effects* are effects applied to a rendered three-dimensional scene that provide additional realism or a stylized look. These effects are often implemented by passing the entire contents of the screen through a pixel shader that applies the desired effect(s). This can be accomplished by rendering a full-screen quad that has been mapped with a texture containing the unfiltered scene. A few examples of full-screen post effects are given below:

- *Motion blur*. This is typically implemented by rendering a buffer of screen-space velocity vectors and using this vector field to selectively blur the rendered image. Blurring is accomplished by passing a *convolution kernel* over the image (see “Image Smoothing and Sharpening by Discrete Convolution” by Dale A. Schumacher, published in [5], for details).
- *Depth of field blur*. This blur effect can be produced by using the contents of the depth buffer to adjust the degree of blur applied at each pixel.
- *Vignette*. In this filmic effect, the brightness or saturation of the image is reduced at the corners of the screen for dramatic effect. It is sometimes implemented by literally rendering a texture overlay on top of the screen. A variation on this effect is used to produce the classic circular effect used to indicate that the player is looking through a pair of binoculars or a weapon scope.
- *Colorization*. The colors of screen pixels can be altered in arbitrary ways as a post-processing effect. For example, all colors except red could be desaturated to grey to produce a striking effect similar to the famous scene of the little girl in the red coat from *Schindler’s List*.

11.5 Further Reading

We’ve covered a lot of material in a very short space in this chapter, but we’ve only just scratched the surface. No doubt you’ll want to explore many of these topics in much greater detail. For an excellent overview of the entire process of creating three-dimensional computer graphics and animation for games and film, I highly recommend [27]. The technology that underlies modern real-time rendering is covered in excellent depth in [2], while [16] is well known as the definitive reference guide to all things related to computer graphics.

Other great books on 3D rendering include [49], [11] and [12]. The mathematics of 3D rendering is covered very well in [32]. No graphics programmer's library would be complete without one or more books from the *Graphics Gems* series ([20], [5], [28], [22] and [42]) and/or the *GPU Gems* series ([15], [44] and [40]). Of course, this short reference list is only the beginning—you will undoubtedly encounter a great many more excellent books on rendering and shaders over the course of your career as a game programmer.